

Cyber Security Internship Project Report

Web Application Vulnerability Scanner

**A Security Assessment Tool for
Detecting Web Vulnerabilities**

Submitted by: Hetal Antala

Date : 25 / 02 / 2026

1 Abstract

This project presents the design and implementation of a Web Application Vulnerability Scanner developed using Python. The scanner automatically crawls web applications, detects input forms, injects test payloads, and identifies common security vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection (SQLi), Cross-Site Request Forgery (CSRF), and missing security headers.

The tool also provides a modern dashboard, risk scoring, severity classification, and downloadable reports in JSON and PDF formats. The system helps security analysts and developers identify security weaknesses efficiently and improve application security posture.

2 Introduction

With the rapid growth of web applications, security vulnerabilities have become a major concern. Attackers exploit weaknesses such as injection flaws, misconfigured headers, and improper authentication mechanisms.

Manual testing is time-consuming and inefficient. Therefore, automated vulnerability scanners are essential for performing security assessments quickly and accurately.

This project aims to build an automated vulnerability scanner similar to tools used in industry.

3 Problem Statement

Web applications often suffer from:

- Cross-Site Scripting (XSS)
- SQL Injection (SQLi)

- Missing security headers
- CSRF attacks
- Poor configuration

Manual identification of these issues is difficult. Hence, there is a need for an automated tool that:

- Crawls websites
- Tests forms
- Injects payloads
- Generates reports

Objectives

The objectives of this project are:

- Develop an automated vulnerability scanner
- Detect XSS, SQLi, CSRF vulnerabilities
- Perform header misconfiguration analysis

- Provide severity-based risk scoring
 - Generate reports in JSON and PDF
 - Build a professional dashboard using Flask
 - Reduce false positives
-

5 Literature Review / Existing Tools

Existing security scanners include:

- OWASP ZAP
- Nikto
- Burp Suite

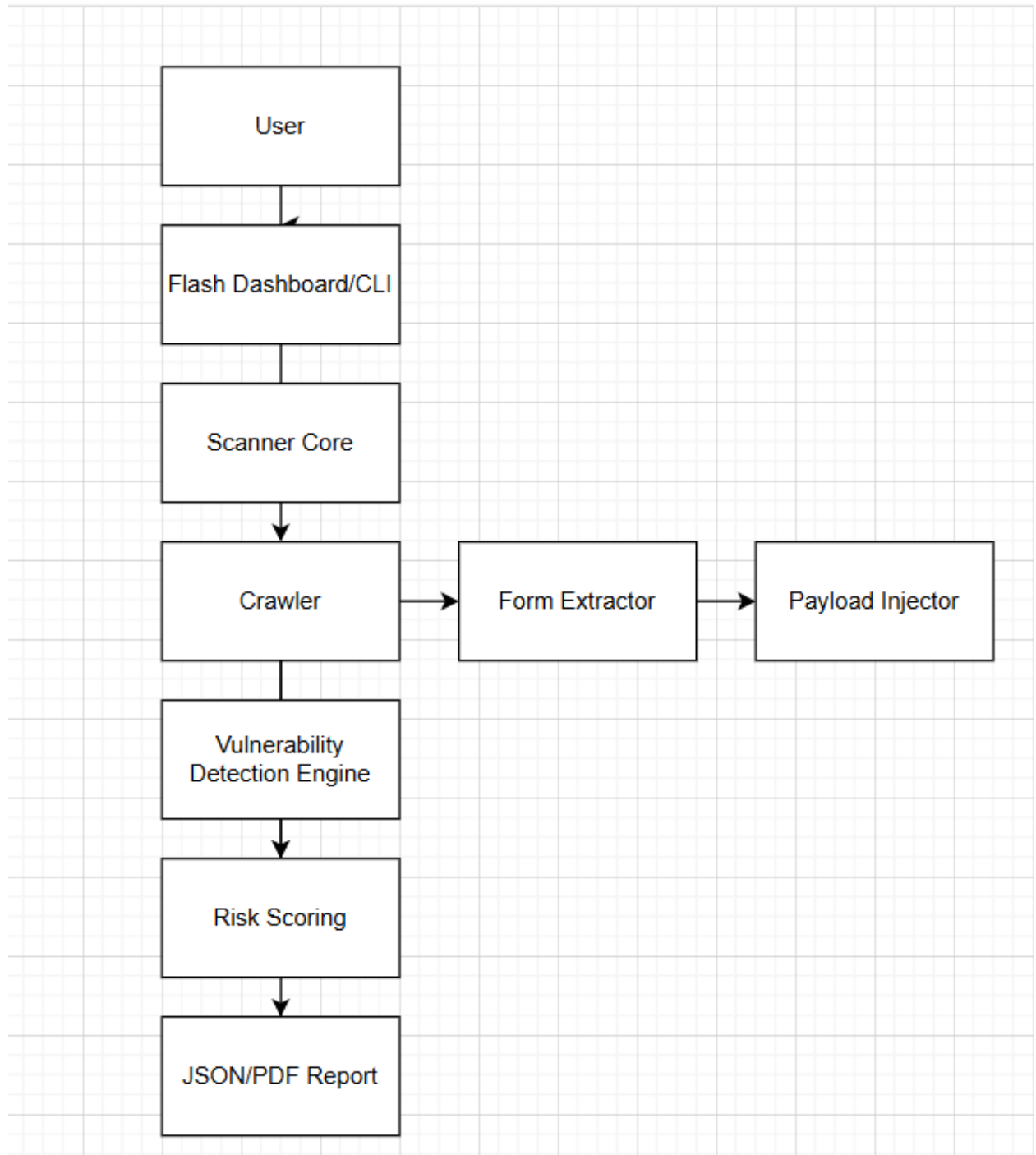
Limitations of these tools:

- Heavyweight
- Complex setup
- Learning curve

This project provides a lightweight educational alternative.

6 System Architecture

Draw this diagram:



7 Technology Stack

Component	Technology
Backend	Python
HTTP	requests
Parsing	BeautifulSoup
Web UI	Flask
Reports	reportlab
Frontend	HTML, CSS
OS	Linux/Windows

8 Methodology

Steps followed:

- 1.Crawl website pages

2.Extract forms and inputs

3.Inject payloads

4.Analyze responses

5.Detect vulnerabilities

6.Assign severity

7.Calculate risk score

8.Generate reports

Modules Description

Crawler

- Discovers URLs
- Extracts forms

Vulnerability Tester

- Injects XSS/SQLi payloads
- Checks response patterns

Header Analyzer

- Detects missing security headers

Risk Engine

- Calculates severity score

Reporter

- Generates JSON and PDF reports

Dashboard

- Displays results visually

10 Detection Logic

XSS Detection

Inject:

`<script>alert(1)</script>`

If reflected → vulnerable

SQL Injection Detection

Inject:

' OR 1=1--

Check SQL error patterns

CSRF Detection

If POST form without token → vulnerable

Header Checks

Verify:

- CSP
- HSTS
- X-Frame-Options

Features Implemented

- ✓ Web crawling
- ✓ Form extraction

- ✓ XSS detection
 - ✓ SQLi detection
 - ✓ CSRF detection
 - ✓ Header checks
 - ✓ Risk scoring
 - ✓ Dashboard UI
 - ✓ JSON reports
 - ✓ PDF reports
 - ✓ Color-coded severity
-

1 2 Testing & Results

Example:

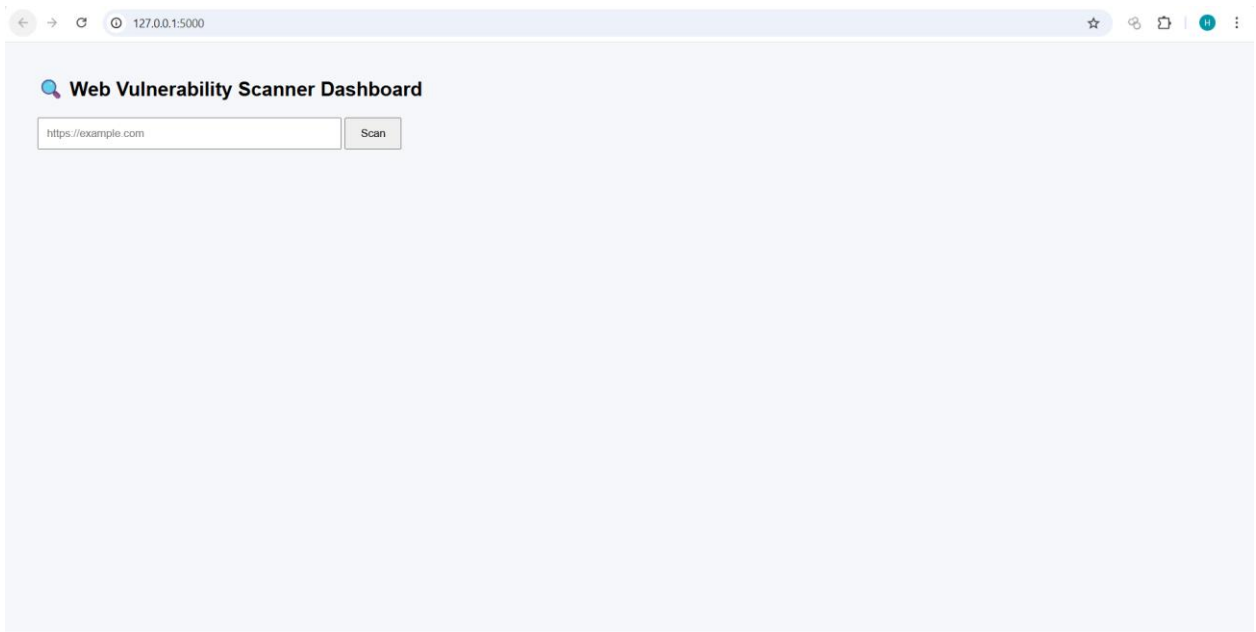
Target: <https://jguni.in>

Results:

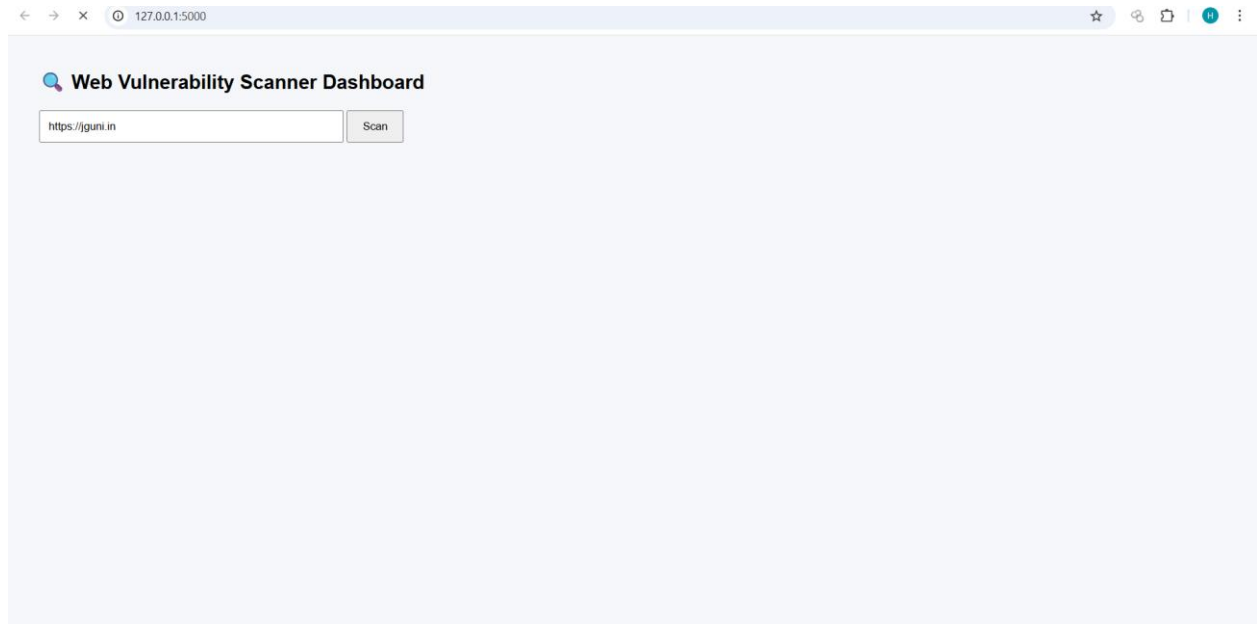
Type	Severity
Missing CSP	Medium
Clickjacking Risk	Medium
HSTS Missing	Low
Risk Score: 7 (Medium)	

1 3 Screenshots

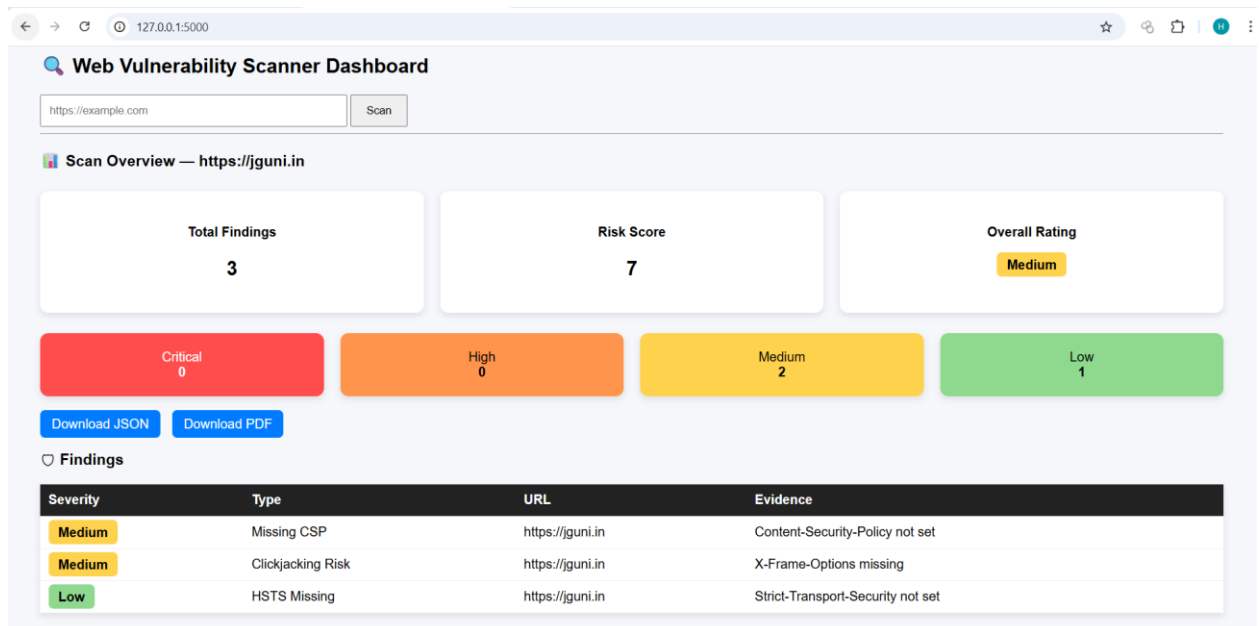
1 Dashboard Home



2 Scan Running



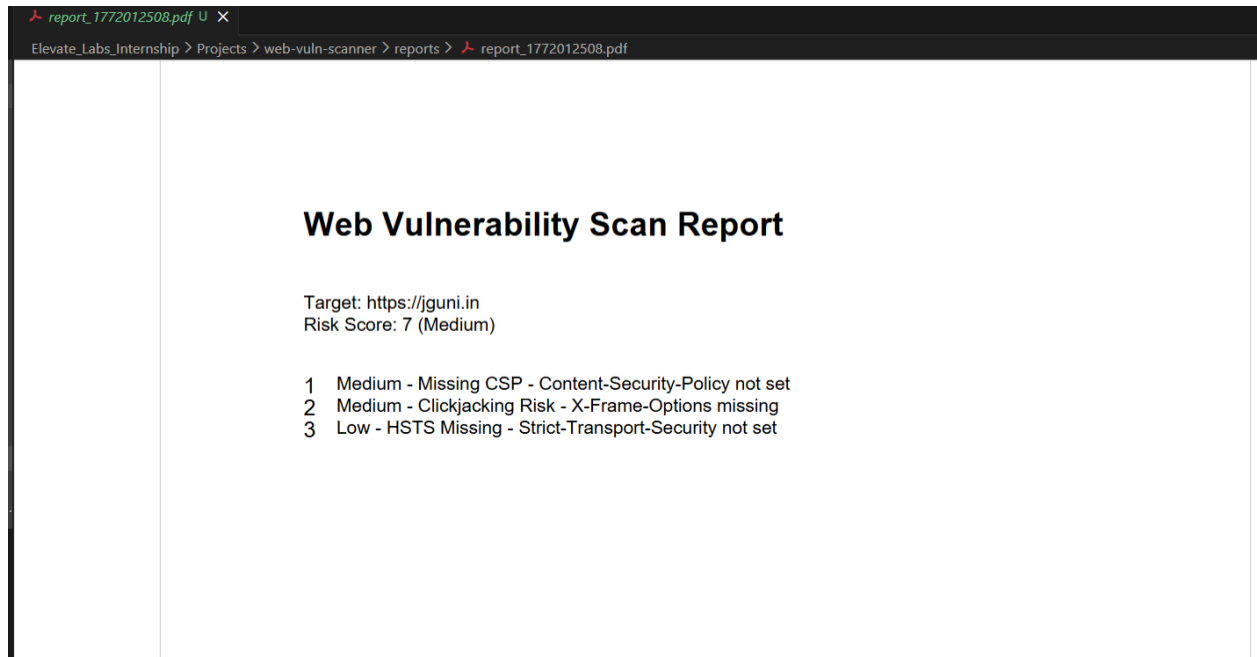
3 Results Page



4 JSON Report

```
$ python run_scanner.py https://jguni.in
[{'target': 'https://jguni.in', 'risk_score': 7, 'rating': 'Medium', 'counts': {'Critical': 0, 'High': 0, 'Medium': 2, 'Low': 1, 'Info': 0}, 'total_findings': 3, 'vulnerabilities': [{'type': 'Missing CSP', 'severity': 'Medium', 'payload': '', 'url': 'https://jguni.in', 'evidence': 'Content-Security-Policy not set'}, {'type': 'Clickjacking Risk', 'severity': 'Medium', 'payload': '', 'url': 'https://jguni.in', 'evidence': 'X-Frame-Options missing'}, {'type': 'HSTS Missing', 'severity': 'Low', 'payload': '', 'url': 'https://jguni.in', 'evidence': 'Strict-Transport-Security not set'}], 'report_file': 'reports/report_1772012508.json', 'pdf_file': 'reports/report_1772012508.pdf'}]
```

5 PDF Report



1 4 Sample Report Output

```
{  
  
  "target": "https://jguni.in/",  
  
  "risk_score": 7,  
  
  "rating": "Medium",
```

```
"counts": {  
  "Critical": 0,  
  "High": 0,  
  "Medium": 2,  
  "Low": 1,  
  "Info": 0  
},  
"total_findings": 3,  
"vulnerabilities": [  
  {  
    "type": "Missing CSP",  
    "severity": "Medium",  
    "payload": "",  
    "url": "",  
    "evidence": "Content-Security-Policy not set"  
  },  
]
```

```
{  
  "type": "Clickjacking Risk",  
  "severity": "Medium",  
  "payload": "",  
  "url": "",  
  "evidence": "X-Frame-Options missing"  
},  
  
{  
  "type": "HSTS Missing",  
  "severity": "Low",  
  "payload": "",  
  "url": "",  
  "evidence": "Strict-Transport-Security not set"  
}  
]  
}
```


1 5 Limitations

- No authenticated testing
- No deep crawling
- No DOM-based XSS detection

Limited payload list

1 6 Future Enhancements

- Login testing
 - Multi-thread scanning
 - Dark mode UI
 - Docker deployment
 - API scanning
 - Machine learning detection
-

1 7 Conclusion

The Web Vulnerability Scanner successfully automates detection of common web vulnerabilities. It provides a lightweight and efficient solution for security assessment. The project enhances understanding of web security concepts and demonstrates practical application of cybersecurity principles.
